Ezoterični programski jezici

Ana-Marija Milin

1 Uvod

Ezoterični(unutarnji, skriven, tajnovit, skrovit) programski jezici su programski jezici osmišljeni u svrhe testiranja granica programiranja, bilo to za dokazivanje njihove učinkovitosti, kao umjetnost, za hakiranje drugog programskog jezika, ili kao šala. Uglavnom, autori ezoteričnih jezika ne stvaraju jezik kako bi se funkcionalno koristio u praktične svrhe. Pa tako ezoterični jezici često nemaju tipičnu jezičnu sintakstu i značajke jezika. Ezoterični programski jezici se poprilično razlikuju, no manje više svi imaju neke zajedničke karakteristike: minimalizam, parodija ili cilj da se programiranje maksimalno oteža. Ne postoji neka strogo definirana raspodjela ezoteričnih programskih jezika, no ipak se javljaju neke značajke po kojima ih možemo grupirati. Minimalni programski jezici(autorima je cilj da broj naredbi bude minimalan), tematski i humoristični jezici(nastali kao inat ili šala na dotadašnje jezike ili djela), vizualni i višedimenzionalni jezici(kod jezika ne prati tipičan "linearan" tok izvršavanja naredbi), te kriptografski(nastali kako bi bili nepotrebno zahtjevni). Ove grupe jezika imaju dosta neobična svojstva, pa se postavlja pitanje jesu li ezoterični jezici uopće pogodni za programiranje i za što ih možemo iskoristiti.

2 INTERCAL pr.Kr.

Programski jezici(ne ezoterični) korišteni do 1972.(godina nastanka prvog ezoteričnog programskog jezika) svakako su služili svojim namjenama. S obzirom da ih ne koristimo u tolikoj mjeri danas, očito su imali neke nepoželjna svojstva. Tako su recimo problemi bili birokratska struktura koda(potrebno pisanje nepotrebnog koda), prevelika interpretabilnost(interpreter ne javlja grešku kada bi trebao), zahtjev programa za specijalnim tipkovnicama(toliko specijalne da ih nemam na tipkovnici), itd. Inspirirani nepotrebnim karakteristikama tadašnjim programskim jezicima, dva studenta su osmislila prvi ezoterični programski jezik: INTERCAL. INTERCAL je bio parodija na tadašnje jezike, te je trebao biti sve što oni nisu, npr. određeni postotak linija koda mora u sebi sadržavati naredbu PLEASE, umjesto GOTO ima COME FROM, itd.

2.1 INTERCAL

Prvi ezoterični jezik INTERCAL, 1972. dizajniran je s ciljem da bude potpuno drugačiji od tadašnjih jezika(FORTRAN, BASIC, COBOL, SNOBOL, LISP). INTERCAL službeno nije bio korišten sve dok 1990. interpreter nije bio implementiran u C-u, nakon čega je interes za INTERCAL-om i ostalim ezoteričnim jezicima porastao. Iz originalnog priručnika:

> Any resemblance of the programming language portrayed here to other programming languages, living or dead, is purely coincidental.

INTERCAL ili Compiler Language With No Pronounceable Acronym je ezoterični jezik osmišljen 1972. godine. Osmislili su ga Don Woods i James M. Lyon kao satiru na tadašnje programske jezike. Trenutno se održavaju samo tri dijalekta INTERCAL-a: C-INTERCAL, CLC-INTERCAL, te INTERCAL64.

INTERCAL ima samo dva tipa varijabli, 16-bitni cijeli broj, reprezentiran s (.), i 32-bitni cijeli broj, reprezentiran s (:), gdje se spremaju nenegativni brojevi od 0 do $2^{16} - 1$ i od 0 do $2^{32} - 1$. Varijable .67 i :67 se razlikuju, dok .00001 i .1 reprezentiraju istu varijablu. Konstante u INTERCAL-u mogu biti isključivo 16-bitne, te ih deklariramo ljestvama. Međutim oprez:

> Caution! Under no circumstances confuse the mesh with the interleave operator, except under confusing circumstances!

INTERCAL prikazuje polja sa ,(16-bitno polje) i ;(32-bitno polje), operacije: umetanje(interleave operator) i odabir(select). Umetanje ¢ kombinira znamenke bitova oba 16-bitna unosa, te kao izlaz daje njihovu 32-bitnu kombinaciju. Primjer: .0¢.65535 daje u bitovima 0101....01 Operator select uzima one bitove prvog operanda čije pozicije odgovaraju pozicijama jedinica drugog operanda. Ako je jedan od operatora veći od 16 bitova, tada je rezultat 32-bitni. Primjer: .179 .201, u bitovima: 10110011 11001001, rezultat: 1001 Unarne operacije su AND(znak za end), OR(V) i XOR(za svaki), te se ne konkateniraju. Jedini način da se pokrene više operacija je grupiranje operacija sa 'ili ". Kako bi se pojednostavnio kod, '. možemo zamijeniti sa !. Labele, odnosno oznake, prikazujemo sa brojem od 0 do 65535 unutar zagrade, s tim da valja imati na umu da su od 1000 do 1999 lokacije rezervirane za biblioteke INTERCAL-a. Nakon labele, ako je uopće ima, slijedi identifier: DO, PLEASE ili PLEASE DO, koje slijedi qualifier: NOT ili N'T(što nulificira naredbu, te kreće izvršavanje programa), i [broj od 0 do 100]% (ovo predstavlja vjerojatnost da se naredba izvrši pri svako prolasku po kodu. Nadalje slijedi jedna od operacija: calculate(jedna od prethodno opisanih operacija), NEXT, FORGET, RESUME, STASH, RETRIEVE, IGNORE, REMEMBER, ABSTAIN, REINSTATE, GIVE UP, INPUT, OUTPUT. Pri računanju ¡- ima funkciju pridruživanja, odnosno jednako.

Primjer greški:

- ICL079 (ili E079) PROGRAMMER IS INSUFFICIENTLY POLITE (PLEASE u manje od 20% linija)
- ICL099 (ili E099) PROGRAMMER IS OVERLY POLITE (PLEASE u više od 33% linija)

3 Možemo li ovo smanjiti?

Kao što je rečeno, uspješna implementacija interpretera INTERCAL-a jako je utjecala na popularizaciju i povećan interes prema ezoteričnim jezicima, kako za sam INTERCAL, tako i za kreaciju novih jezika. Polako su nastajali novi ezoterični jezici koji bi imali razne karakteristike. Jedan od smjerova bio je minimalistički pristup, gdje je cilj smisliti što manji mogući jezik. Jedan od prvih minimalnih ezoteričnih jezika bio je FALSE(nazvan po autorovoj najdražoj istinitonoj vrijednosti). Cilj je bio stvoriti što jači, konfuzniji jezik sa što manjim prevoditeljem. Originalni prevoditelj imao je 1024 bajta i bio je napisan u asembleru za M68k. Već iste godine, inspiriran FALSE-om, nastao je brainfuck, reprezentant u klasi minimalnih ezoteričnih jezika.

3.1 brainfuck

Brainfuck osmislio je Urban Müller 1993. Ideja iza jezika bila je mogućnost pisanja najmanjeg prevoditelja. Inspiriran je FALSE-om koji je imao 1024-bajtni prevoditelj, dok je za brainfuck napisao 240-bajtni prevoditelj. Dao mu je ime brainfuck, međutim zbog ime se često piše Brainfuck, Brainf***, Brainf*ck, brainf***, brainflakes, bf...zbog čega može biti teško skupiti svu dostupnu literaturu. brainfuck radi nad poljem memorijskih ćelija, na početku inicijaliziranih na nulu. Memorija svakako ovisi o interpreteru. Pointer inicijalno pokazuje na prvu ćeliju u memoriji. brainfuck kao reprezentant minimalizma, ima samo 8 naredbi:

- Move the pointer to the right
- Move the pointer to the left
- Decrement the memory cell at the pointer
- increment the memory cell at the pointer
- Output the character signified by the cell at the pointer
- Input the character signified by the cell at the pointer
- Input a character and store it in the cell at the pointer
- Jump past the matching] if the cell at the pointer is 0
- Jump back to the matching [if the cell at the pointer is nonzero

Dakle, vidimo da brainfuck stvarno je minimalan jezik. Međutim, to ne znači da u njemu ne možemo rješavati ozbiljne probleme. Radna traka je u svakom trenutku konačna(ima konačno mnogo nepraznih ćelija), pa je možemo kodirati kao konačan niz znakova. Uz dovoljno fizički osigurane memorije, bf ima beskonačno mnogo praznih ćelija. Ćelije drže nenegativne brojeve, te ¡ i ¿ pomiču pokazivač. Simboli na traci i stanja su brojevi, te tablica prijelaza prikazuje promjenu stanja kada uđemo u petlju pomoću [i], što se može dogoditi beskonačno mnogo puta. Dakle, pročita se simbol, obavlja se instrukcija i mijenja se stanje. Budući da za svaki Turingov stroj postoji program u brainfucku koji mu simulira ponašanje, brainfuck je Turing potpun. Turing potpunost i manjak ugrađenih naredbi, čine brainfuck reprezentativan primjer iz obitelji Turing tarpita/Turingovih močvara/Turingov ponor. Turing tarpit je programski jezik koji je Turing potpun i ima minimalan broj operacija. Inače, neki ezoterični programski jezici, namjerno su ne-Turing potpuni.

4 Kod ne mora biti linija

Jedan od smjerova ezoteričnih jezika je bio minimalizam(navedena je i Turing nepotpunost). Međutim, to nije jedini način kretanja razvoja ezoteričnih jezika. Primjetimo kako smo u bf limitirani sa načnom kretanja kroz prostor u memoriji, lijevo i desno po jednoj traci memorije. Takav program je poprilično lagan za prevođenje u strojni kod(ali težak za ljude). Ezoterični jezik Befunge je nastao upravo kao suprotnost tim idejama. Autor ga je napisao kako bi bio što teži za prevesti u strojni kod. Tok programa je 2D, umjesto tipičnog niza instrukcija od gore prema dolje, pokazivač instrukcija se kreće u svim smjerovima po mreži. Koristeći instrukcije **put** i **get**, program može u stvarnom vremenu mijenjati vlastite znakove na mreži dok se izvršava. To je jako teško za tipične prevoditelje.

4.1 Befunge

Befunge je dvodimenzionalni ezoterični programski jezik osmišljen 1993. s ciljem da bude što teži za izvršavanje. Ime je nastalo tako da je autor Chris Pressey u 4 ujutro krivo napisao before(prije/prethodno). Kod se nalazi na dvodimenzionalnoj mreži instrukcija(ASCII charactera koje reprezentiraju instrukciju), te se izvedba programa može odvijati u bilo kojem smjeru na mreži, konkretno: gore, dolje, lijevo, desno. Smatra se da je prvi 2D programski jezik. Prvi u Funge obitelji je Befunge−93, dok mu je Befunge−98 najsličniji.

Izvedba programa se odvija programskim brojačem(−93) ili instrukcijskim pokazivačem(−98). Brojač, odnosno pokazivač pokazuje na polje u mreži igrališta(playfield). Instrukcijski pokazivač ima inerciju/tromost, može ići u bilo kojem smjeru dok ne naiđe na polje sa instrukcijom zbog koje mora promijeniti smjer. Instrukcijski pokazivač kreće u gornjem lijevom kutu IGRALIŠTA, te se kreće desno, sve dok ne naiđe na instrukciju zbog koje mora promijeniti smjer. Kako nailazimo na nove instrukcije one se izvode. Befunge−93 može preskočiti do dvije ćelije, dok Befunge−98 može dalje, međutim zbog inercije ne koriste se često.

Befunge je 2D, međutim to nije jedina dimenzionalnost koju Funge obitelj pruža. Unefunge je 1D, Trefunge je 3D, Nefunge ND...

U Befungu je napisan kod za Fungot, bot čiji kod je tako mali da stane kao print na majci.

Befunge−93 ima limit na 80x25 ćelija na igralištu, te programi imaju radni stog sa 32-bitnim elementima. Postoje potisni automati koji ne mogu biti simulirani u Befunge−93. Srećom, pa Befunge−98 nema takve restrikcije, pa je Turing potpun.

5 Kod ne mora izgledati kao kod

Do sada smo prošli neke neobične jezike, ali svi koliko god neobični bili, podsiječaju na kod. Međutim, ne mora tako uvijek biti, bar ne ako se pita autore sljedećih jezika, a i mnoge druge. Programski jezik Piet nastao je, jer je autor jezika bio inspiriran djelima Pieta Mondreina, te je htio dizajnirati jezik koji kao funkcionalan kod, izgleda i kao apstraktna umjetnost. Dok su autori Whitespace-a stvorili jezik kao bunt na ignorirajuće bijeline u kodu, te se u Whitespace-u ignorira samo tekst. Nadalje, jezik Chef kao kod mora imati primjenjiv recept.

5.1 Piet

Programski jezik Piet, dizajnirao je David Morgan-Mar s ciljem da programi izgledaju kao apstraktna vizualna umjetnost. Inspiracija su bile slike Pieta Mondraina, te se jezik originalno trebao zvati Mondrain, međutim ime je već bilo zauzeto.

> I would have liked to call the language Mondrian, but someone beat me to it with a rather mundane-looking scripting language. Oh well, we can't all be esoteric language writers I suppose.

Piet koristi 20 različitih boja. Njih 18 su zatvorene u cikličan krug, zapravo se radi o 6 boja(crvena, žuta, zelena, cijan, plava i magenta) i njihovim svijetlinama(svijetla, intenzivna i tamna). Dodatne dvije boje, bijela i crna, ne pripadaju tom zatvorenom krugu. Krug ide po bojama crvena-¿žuta-¿...-¿magenta-¿crvena, te svijetlinama, svijetla-¿intenzivna-¿tamna-¿svijetla. Ostale boje se također mogu koristiti, no to ovisi o interpreteru. Standardni interpreter tretira takve boje kao bijelu.

U Pietu gradivne jedinice su blokovi boja. Blok boja je više 'piksela', odnosno kodela iste boje koji se dodiruju nedijagonalno, te može sadržavati praznine ili druge boje unutar sebe. Svi podatci u Pietu se spremaju na stog kao cijeli brojevi. Broj kodela u necrnom ili nebijelom bloku predstavlja cijeli broj, koji se može ili ne mora spremiti na stog. Negativni brojevi se ne mogu reprezentirati kodelima, ali se mogu generirati operacijama. Konačnost/beskonačnost Piet-a ovisi o interpreteru. Crni kodeli i rub označava kraj promjenu smjera. Izvršavanje programa kreće u gornjem lijevom kutu. Interpreter čuva Pokazivač smjera i Izbornik kodela. Smjerovi Pokazivača smjera su gore, dolje, lijevo, desno(početno),a Izbornika kodela desno i lijevo(početno). Interprter se kreće u smjeru koji nalaže Pokazivač smjera, te se kreće prema rubnom kodelu. Kada stigne do ruba, Izbornik kodela daje informaciju o tome koji kodel je potrebno uzeti, te interpreter nastavlja u novi blok. Interpreter će rotirati Pokazivača smjera i Izbornika kodela ako se ne može maknuti iz bloka, no ako nakon osme rotacije ne može van, program se zaustavlja. Bijeli kodeli su slobodne zone, preko njih Pokazivač smjera prolazi do sljedećeg bloka druge boje bez izvršavanja naredbi.

5.2 Whitespace

Whitespace, 2002. osmislili su Edwin Brady i Chris Morris. Inspiracija iz jezika, bilo je ne korištenje bijelina u tipičnim programskim jezicima. Za razliku od tipičnih programskih jezika, bar u to vrijeme, gdje se bijeline ignoriraju, Whitespace ignorira sve ostalo što nisu bijeline. Time, u Whitespace-u prostor između bijelina možemo popuniti kodom nekog drugog jezika(odnosno u drugom programskom jeziku pazimo na bijeline), što čini takav kod poliglotskim. Dakle, Whitespace koristi bijeline, odnosno, SPACE, ENTER i LEFT TAB, te se njihove kombinacije interpretiraju kao različite instrukcije. Ako naredba počinje sa SPACE, znači da se radi o manipulaciji stoga, što se događa sa stogom ovisi o naredbama nakon. Ako naredba počinje sa TAB SPACE, izvršava se aritmetička instrukcija. ENTER mijenja tok izvršavajna naredbi, TAB ENTER služi za unos i izlaz...

5.3 Chef

Chef je programski jezik gdje programi izgledaju kao kulinarski recepti. Dizajnirao ga je David Morgan-Mar 2002. godine. Principi dizajniranja

- Programski recepti, osim što moraju davati validan izlaz, moraju biti ukusni i jednostavni za pripremu
- Recepti mogu odgovarati kuharima sa različitim budžetima
- Recepti koriste metrički sustav, ali mogu sadržavati tradicionalne kulinarske mjere, kao što su šalica i žlica

Svi recepti imaju sastojke, pa Chef koristi sastojke za pohranu podataka. Svi sastojci su numerički, ali mogu se interpretirati i kao Unicode. Tekući sastojci su izlazni Unicode simboli, a suhi i nespecificirani sastojci su izlazni numerički podaci. Kuhar ima na raspolaganju neograničenu zalihu posuda za miješanje i posuda za pečenje. One mogu sadržavati vrijednosti sastojaka. Sastojci su u posudama poredani u stog. Ako se vrijednost sastojka promijeni, vrijednost u posudi ostaje nepromjenjena. Vrijednosti u posudama imaju oznake suho ili tekuće. Ako želimo pozvati neku od posuda

moramo se refereirati na nju sa rednim brojem(1st, 2nd, 3rd, 10th). Ako imamo samo jednu posudu, redni broj nije potreban. Recept počinje sa naslovom koji kombinira ono što program radi i jelo koje se može pripremiti, npr. Fibbonaccijevi brojevi s ajvarom od banane. Zatim nabrajamo satojke i njihove količine, odnosno naznačimo količinu i mjeru(npr. 10g) koje nam kažu vrijedost, te njihovu značajku(mokro, suho, nedefinirano). Vrijeme kuhanja i temperatura pečnice su brojevi. Sastojci, vrijeme i temperatura su opcionalni. Neke od metoda:

- Take ingredient from refrigerator
- Put ingredient into [nth] mixing bowl.
- Fold ingredient into [nth] mixing bowl.
- Add ingredient [to [nth] mixing bowl].
- Remove ingredient [from [nth] mixing bowl].

6 Koliko ovo teško može biti?

Mnogi ezoterični jezici kao glavni fokus imaju nekakvu zanimljivu motivaciju za nastanak(minimalizam, iskorištavanje do tada ne korištenih mogućnosti, estetska privlačnost...) Neki od njih kao motivaciju za nastanak imaju: kako maksimalno otežati stvari? Pa, sigurno postoji najteži. Kralj među njima, Malbolge nastao je isključivo s jednim ciljem, rad u jeziku u čijem je okruženju rad nemoguć.

6.1 Malbolge

Malbolge je programski jezik kojeg je 1998. osmislio Ben Olmstead s ciljem da to bude najteži programski jezik na svijetu. Pa ga je u tom duhu i nazvao Malebolge, osmi krug Pakla iz Božanstvene komedije. Originalno je planirano da je u jeziku gotovo nemoguće raditi, čak ni sam autor jezika ne zna raditi u Malbolgu. Prvi Malbolge program, Hello world, generiran je pretraživanjem uz pomoć genetskog algoritma,a za to mu je trebalo dvije godine.

Malbolge radi s brojevima u bazi 3, a za adresiranje i aritmetiku koristi crazy operaciju, tablica 3x3 trita. Pa se ne može koristiti standardna intuicija o zbrajanju ili množenju. Nakon što je znak pročitan, mijenja se funkcijom crypt, te se mijenja dok se izvršava. Pa, pri svakom prolasku isti redak može davati različite instrukcije, što čini program praktički nečitljiv. Operacija koja se izvršava ne čita se direktno iz instrukcijskog pokazivača, vrijednost znaka c prolazi kroz formulu (c+pokazivac)mod 94. Nadalje, program mora biti unaprijed kriptiran da bi opće bio validan, inače se ne izvršava. Kontrola toka prolazi kroz iste mehanizme permutacije. Vrijednost s kojima radimo žive u istom prostoru sa instrukcijama, pa pokazivač može čitati podatak kao instrukciju. Zbog tog samomodificirajućeg koda, javljaju se ideje da se Malbolge koristi u kriptografske svrhe, iako za sada ta ideja nije realizirana.

7 Zašto ezoterični programski jezici?

S obzirom da ezoterični jezici nisu optimalan način programiranja, imaju li uopće smisla? Svakako, velikom većinom se javljaju kao odgovor na tadašnji način programiranja, ili barem aktualne trendove u programiranju. Uglavnom su originalno šale, te zbog svojih ideja nisu baš uporabljivi. Unatoč tome što su većinom rađeni kao eksperimentiranje sa programiranjem, ne može se baš reći da su u potpunosti beskorisni. Reći da je rad u nekom ezoteričnom jeziku beskoristan je kao da kažemo da je bavljenje fizičkom aktivnosti beskorisno. Pojedine značajke koje ezoterični jezici imaju su otkrile neke dotadašnje nedostatke o kojima se počelo voditi računa u kasnijim verzijama, te su ezoterični jezici bili iskorišteni na konkretan način. Npr, JSFuck(nešto kao bf) je bio iskorišten za provale. JSFuck ima 6 naredbi, a kako Firewall traži riječi u kodu kao što su **script** i **alert**, JSFuck je teško uhvatiti. Nadalje, Malbolge, zbog samo generirajućeg koda se potencijalno može iskoristiti u kriptografiji, s obzirom da programima očito trebaju godine da uopće generiraju Malbolge kod. Whitespace je svojom pojavom pokazao kako programski jezici ne koriste optimalno praznine, kojima je moguće dodatno manipulirati.

Literatura

- INTERCAL, 3e8.org
- Piet, David Morgan-Mar, dangermouse.net
- Rad o Malbolgeu, IPSJ SIG Programming 2014
- Malbolge, esolangs.org
- Whitespace, Wikipedia
- The Brainfuck Programming Language Tutorial
- Befunge, esolangs.org
- Befunge-93 specifikacija, nsl.com
- Chef, esolangs.org